EC31203: Communication-I
Experiment: Monte Carlo Simulation of FM in AWGN Channel

Jai Raj
23EC10031
Instructor: Dr. Goutam Das

Department of E&ECE
IIT Kharagpur

Date of Experiment: 12th *November* 2025

1 Aim of the Experiment

The objectives of this experiment are:

1. To simulate an FM communication system using Monte Carlo analysis and evaluate the relationship between Figure of Merit (FOM) and channel noise spectral density.
2. To study the effect of pre-emphasis and de-emphasis filtering on FM noise performance.
3. To analyse the variation of FOM with frequency sensitivity K_f for systems with and without emphasis filtering.

2 Theory

2.1 Frequency Modulation

In Frequency Modulation (FM), the instantaneous frequency of the carrier varies proportionally with the message signal $m(t)$:

$$\omega_i(t) = \omega_c + k_f m(t)$$

The FM signal is:

$$s(t) = A_c \cos\left(\omega_c t + 2\pi K_f \int_{-\infty}^{t} m(\tau)\, d\tau\right)$$

Here,

- A_c = carrier amplitude
- ω_c = carrier angular frequency
- K_f = frequency sensitivity (Hz/V)

2.2 Figure of Merit (FOM)

The system performance is measured using the Figure of Merit:

$$FOM = \frac{(SNR)_{\text{out}}}{(SNR)_{\text{in}}}$$

Higher FOM means better noise suppression.

2.3 Pre-Emphasis and De-Emphasis

FM demodulation inherently boosts high-frequency noise due to differentiation. To reduce this, a pre-emphasis filter is applied:

$$H_{pe}(f) = 1 + j\frac{f}{f_0}$$

with $f_0 = 15\,\text{kHz}$.

At the receiver, the reverse operation is done using de-emphasis:

$$H_{de}(f) = \frac{1}{1 + j\frac{f}{f_0}}$$

These filters boost high-frequency components before transmission and attenuate high-frequency noise after demodulation.

3 Simulation Setup

- Message signal: 1 kHz sinusoid, 2V peak-to-peak
- Carrier frequency: 100 MHz
- Sampling frequency: 205 MHz
- K_f: 300 kHz/V to 800 kHz/V
- Noise: AWGN with PSD from -180 dBm/Hz to -170 dBm/Hz
- IF bandwidth: 2 MHz
- Monte Carlo trials: 100

4 MATLAB Code

4.1 Task 1 Code: FOM vs Noise (Without Pre-Emphasis)

```
% ---- 1. System Parameters ----
fm = 1e3;
Am = 1.0;
Ac = 1.0;
Kf = 500e3;
```

```
% ---- 2. Simulation Parameters ----
delta_f = Kf * Am;
BT = 2 * (delta_f + fm);
Fs = BT * 2.5;
dt = 1/Fs;
Tsim = 100 / fm;
t = (0:dt:Tsim-dt)';
L_sig = length(t);

fprintf('--- Simulation Setup ---\n');
fprintf('Modulation Index (beta): %.1f\n', delta_f/fm);
fprintf('Carson Bandwidth (BT): %.2f MHz\n', BT/1e6);
fprintf('Sampling Frequency (Fs): %.2f MHz\n', Fs/1e6);

% ---- 3. Generate Signals ----
m_t = Am * cos(2*pi*fm*t);
phi_t = 2*pi*Kf * cumsum(m_t) * dt;
s_bb = Ac * exp(1j * phi_t);
Pc = Ac^2;

% ---- 4. Demodulator LPF (Toolbox-Free) ----
k = round(0.44 * Fs / fm);
if mod(k, 2) == 0, k = k + 1; end
fprintf('Using a %d-tap Moving Average filter.\n', k);
lpf_b = (1/k) * ones(1, k);
lpf_a = 1;
transient_samples = round((k-1) / 2);

% ---- 5. Get Clean Output Signal Power (Ps_out) ----
y_phase_clean = unwrap(angle(s_bb));
y_demod_clean_unscaled = [0; diff(y_phase_clean)] / dt;
y_demod_clean = (1 / (2*pi*Kf)) * y_demod_clean_unscaled;
y_filtered_clean = filter(lpf_b, lpf_a, y_demod_clean);
y_clean_steadystate = y_filtered_clean(transient_samples+1 : end);
Ps_out = var(y_clean_steadystate);
fprintf('Calculated Output Signal Power (Ps_out): %.4e W\n', Ps_out);
fprintf('-----------------------------------------\n');

% ---- 6. Monte Carlo Loop ----
psd_dBm_per_Hz = -180:1:-170;
num_points = length(psd_dBm_per_Hz);
FOM_dB = zeros(num_points, 1);
SNR_out_dB = zeros(num_points, 1);
SNR_in_dB = zeros(num_points, 1);

fprintf('Running Monte Carlo Simulation...\n');
for i = 1:num_points
    current_psd_dBm = psd_dBm_per_Hz(i);
    current_psd_dBW = current_psd_dBm - 30;
    N0_over_2_W_Hz = 10^(current_psd_dBW / 10);

    N0 = 2 * N0_over_2_W_Hz;
    Pn_in = N0 * BT;

    SNR_in = Pc / Pn_in;
    SNR_in_dB(i) = 10*log10(SNR_in);

    noisePower = N0 * Fs;
    sigma = sqrt(noisePower / 2);

    noise_I = sigma * randn(L_sig, 1);
    noise_Q = sigma * randn(L_sig, 1);
    complex_noise = noise_I + 1j * noise_Q;
```

```
    r_bb_noisy = s_bb + complex_noise;

    y_phase_noisy = unwrap(angle(r_bb_noisy));
    y_demod_noisy_unscaled = [0; diff(y_phase_noisy)] / dt;
    y_demod_noisy = (1 / (2*pi*Kf)) * y_demod_noisy_unscaled;
    y_filtered_noisy = filter(lpf_b, lpf_a, y_demod_noisy);

    y_total_steadystate = y_filtered_noisy(transient_samples+1 : end);
    y_noise_component = y_total_steadystate - y_clean_steadystate;

    Pn_out = var(y_noise_component);

    SNR_out = Ps_out / Pn_out;
    SNR_out_dB(i) = 10*log10(SNR_out);

    FOM_sim = SNR_out / SNR_in;
    FOM_dB(i) = 10 * log10(FOM_sim);

    fprintf('PSD: %d dBm/Hz | SNR_in: %.1f dB | SNR_out: %.1f dB | FOM: %.2f dB\n
        ', ...
            current_psd_dBm, SNR_in_dB(i), SNR_out_dB(i), FOM_dB(i));
end

% --- 7. Plot Results ---
figure;
plot(psd_dBm_per_Hz, FOM_sim, 'bo-', 'LineWidth', 2);
title('FM Figure of Merit (FOM) vs. Noise PSD');
xlabel('Noise PSD (N_0/2) [dBm/Hz]');
ylabel('FOM (SNR_{out} / SNR_{in})');
grid on;
set(gca, 'FontSize', 12);
ylim([min(FOM_sim)-2, max(FOM_sim)+2]);
```

4.2 Task 2 Code: With Pre-Emphasis / De-Emphasis

```
%
%-------------------------------------------------------
% Simulation Constants
%-------------------------------------------------------
f_break = 15e3;
Fs = 2.05e8;
Tsim = 0.01;
tAxis = 0:1/Fs:Tsim;
f_msg = 1e3;
f_carr = 100e6;
A_carr = 1;
Kf_dev = 5e5;
Vsig_pp = 2;
runs = 10;
BW_IF = 2e6;

%-------------------------------------------------------
% Message & Pre-Emphasis
%-------------------------------------------------------
msg = (Vsig_pp/2) * sin(2*pi*f_msg*tAxis);

w0 = 2*pi*f_break;
tau_f = 1/w0;
coeff = exp(-1/(Fs*tau_f));
```

```
msg_pre = filter([1 -coeff], [1], msg);

%-------------------------------------------------------------
% FM Modulation
%-------------------------------------------------------------
phi = 2*pi*Kf_dev * cumtrapz(tAxis, msg_pre);
s_tx = A_carr * cos(2*pi*f_carr*tAxis + phi);

%-------------------------------------------------------------
% Noise Spectral Setup
%-------------------------------------------------------------
Noise_dBmHz = -180:1:-170;
Noise_WHz = 10.^(Noise_dBmHz/10) * 1e-3;

SNRin_lin = zeros(size(Noise_WHz));
SNRout_lin = zeros(size(Noise_WHz));
FOM_metric = zeros(size(Noise_WHz));

%-------------------------------------------------------------
% Reference Demodulation (Noise-Free)
%-------------------------------------------------------------
s_tx_bp = bandpass(s_tx, [f_carr - BW_IF/2, f_carr + BW_IF/2], Fs);
phase_ref = unwrap(angle(hilbert(s_tx_bp)));
demod_ref = diff(phase_ref) * Fs / (2*pi*Kf_dev);
demod_ref = demod_ref - f_carr/Kf_dev;
demod_ref = filter([1], [1 -coeff], demod_ref);
demod_ref = lowpass(demod_ref, 2.5e3, Fs);

P_sig = var(demod_ref);

%-------------------------------------------------------------
% Monte Carlo Evaluation
%-------------------------------------------------------------
for k = 1:length(Noise_WHz)

    N0 = Noise_WHz(k);
    noise_var_total = N0 * Fs / 2;
    noise_IF_power = N0 * BW_IF;
    Pin = (A_carr^2) / 2;
    SNRin_lin(k) = Pin / noise_IF_power;

    SNRtemp = zeros(1, runs);

    for trial = 1:runs
        n = sqrt(noise_var_total) * randn(size(s_tx));
        rcv = s_tx + n;

        rcv_bp = bandpass(rcv, [f_carr - BW_IF/2, f_carr + BW_IF/2], Fs);

        phase_rcv = unwrap(angle(hilbert(rcv_bp)));
        demod = diff(phase_rcv) * Fs / (2*pi*Kf_dev);
        demod = demod - f_carr/Kf_dev;
        demod = filter([1], [1 -coeff], demod);
        demod = lowpass(demod, 2.5e3, Fs);

        Pnoise = var(demod - demod_ref);
        SNRtemp(trial) = P_sig / Pnoise;
    end

    SNRout_lin(k) = mean(SNRtemp);
    FOM_metric(k) = SNRout_lin(k) / SNRin_lin(k);
end
```

```
%-------------------------------------------------------------
% Plot Results
%-------------------------------------------------------------
figure;
plot(Noise_dBmHz, FOM_metric, 'r-s', 'LineWidth', 1.3);
xlabel('Noise Spectral Density (dBm/Hz)');
ylabel('Figure of Merit (FOM)');
title('FM System with Pre-/De-Emphasis: FOM vs. Noise Density');
grid on;
set(gca, 'FontSize', 10);
```

4.3 Task 3.1 Code: FOM vs Kf (Without Emphasis)

```
clear; clc; close all;

%-------------------------------------------------------------
% Simulation Parameters
%-------------------------------------------------------------
Fs_rate = 2.05e8;
Tsim = 0.01;
timeVec = 0:1/Fs_rate:Tsim;
fm_base = 1e3;
fc_main = 100e6;
A_car = 1;
kf_range = 1e5:5e4:8e5;
Vpp_in = 2;
MC_trials = 10;
BW_IF = 2e6;

%-------------------------------------------------------------
% Channel Noise Parameters
%-------------------------------------------------------------
NoisePSD_dBm = -175;
NoisePSD_lin = 10^(NoisePSD_dBm/10) * 1e-3;

FOM_metric = zeros(size(kf_range));
SNR_in_lin = zeros(size(kf_range));
SNR_out_lin = zeros(size(kf_range));

%-------------------------------------------------------------
% Monte Carlo Simulation
%-------------------------------------------------------------
for idx = 1:length(kf_range)
    kf_now = kf_range(idx);

    msg = (Vpp_in/2) * sin(2*pi*fm_base*timeVec);
    phaseMod = 2*pi*kf_now * cumtrapz(timeVec, msg);
    s_fm = A_car * cos(2*pi*fc_main*timeVec + phaseMod);

    s_fm_if = bandpass(s_fm, [fc_main - BW_IF/2, fc_main + BW_IF/2], Fs_rate);
    phase_ref = unwrap(angle(hilbert(s_fm_if)));
    ref_demod = diff(phase_ref) * Fs_rate / (2*pi*kf_now);
    ref_demod = ref_demod - fc_main/kf_now;
    ref_demod = lowpass(ref_demod, 2.5e3, Fs_rate);
    P_signal = var(ref_demod);

    N_total = NoisePSD_lin * Fs_rate/2;
    N_IF = NoisePSD_lin * BW_IF;
```

```
    P_car = A_car^2 / 2;
    SNR_in_lin(idx) = P_car / N_IF;

    trial_SNR = zeros(1, MC_trials);

    for run = 1:MC_trials
        noiseVec = sqrt(N_total) * randn(size(s_fm));
        rx_signal = s_fm + noiseVec;

        rx_bp = bandpass(rx_signal, [fc_main - BW_IF/2, fc_main + BW_IF/2],
            Fs_rate);

        phase_rx = unwrap(angle(hilbert(rx_bp)));
        demod = diff(phase_rx) * Fs_rate / (2*pi*kf_now);
        demod = demod - fc_main/kf_now;
        demod = lowpass(demod, 2.5e3, Fs_rate);

        noise_var = var(demod - ref_demod);
        trial_SNR(run) = P_signal / noise_var;

    end

    SNR_out_lin(idx) = mean(trial_SNR);
    FOM_metric(idx) = SNR_out_lin(idx) / SNR_in_lin(idx);

end

%-------------------------------------------------------------
% Plotting
%-------------------------------------------------------------
figure;
plot(kf_range, FOM_metric, 'm-d', 'LineWidth', 1.3);
xlabel('Frequency Sensitivity K_f (Hz/V)');
ylabel('Figure of Merit (FOM)');
title('FM Variation with K_f (No Pre-/De-Emphasis)');
grid on;
set(gca, 'FontSize', 10);
```

4.4 Task 3.2 Code: FOM vs Kf (With Emphasis)

```
clear; clc; close all;

%-------------------------------------------------------------
% Simulation Constants
%-------------------------------------------------------------
f_cutoff = 15e3;
Fsamp = 2.05e8;
Tduration = 0.01;
tAxis = 0:1/Fsamp:Tduration;
f_base = 1e3;
f_carrier = 100e6;
A_carrier = 1;
kf_span = 1e5:1e5:8e5;
Vpeak2peak = 2;
NMonte = 10;
BW_IF = 2e6;

%-------------------------------------------------------------
% Channel Noise Parameters
%-------------------------------------------------------------
```

```
Noise_dBmHz = -175;
Noise_WHz = 10^(Noise_dBmHz/10) * 1e-3;

FOM_val = zeros(size(kf_span));
SNR_inlin = zeros(size(kf_span));
SNR_outlin = zeros(size(kf_span));

%-------------------------------------------------------
% Pre-Emphasis Filter Design
%-------------------------------------------------------
omega0 = 2*pi*f_cutoff;
tau_c = 1/omega0;
beta = exp(-1/(Fsamp*tau_c));

%-------------------------------------------------------
% Monte Carlo Simulation
%-------------------------------------------------------
for m = 1:length(kf_span)

    kf_now = kf_span(m);

    msg_wave = (Vpeak2peak/2) * sin(2*pi*f_base*tAxis);
    msg_pre = filter([1 -beta], [1], msg_wave);

    phi_dev = 2*pi*kf_now * cumtrapz(tAxis, msg_pre);
    s_tx = A_carrier * cos(2*pi*f_carrier*tAxis + phi_dev);

    s_tx_bp = bandpass(s_tx, [f_carrier - BW_IF/2, f_carrier + BW_IF/2], Fsamp);
    ref_phase = unwrap(angle(hilbert(s_tx_bp)));
    ref_demod = diff(ref_phase) * Fsamp / (2*pi*kf_now);
    ref_demod = ref_demod - f_carrier/kf_now;
    ref_demod = filter([1], [1 -beta], ref_demod);
    ref_demod = lowpass(ref_demod, 2.5e3, Fsamp);
    P_sig = var(ref_demod);

    noise_var_full = Noise_WHz * Fsamp/2;
    noise_power_IF = Noise_WHz * BW_IF;
    P_in = A_carrier^2 / 2;
    SNR_inlin(m) = P_in / noise_power_IF;

    TempSNR = zeros(1, NMonte);

    for r = 1:NMonte
        n = sqrt(noise_var_full) * randn(size(s_tx));
        rcv = s_tx + n;

        rcv_bp = bandpass(rcv, [f_carrier - BW_IF/2, f_carrier + BW_IF/2], Fsamp)
            ;

        rcv_phase = unwrap(angle(hilbert(rcv_bp)));
        demod_out = diff(rcv_phase) * Fsamp / (2*pi*kf_now);
        demod_out = demod_out - f_carrier/kf_now;
        demod_out = filter([1], [1 -beta], demod_out);
        demod_out = lowpass(demod_out, 2.5e3, Fsamp);

        noise_var = var(demod_out - ref_demod);
        TempSNR(r) = P_sig / noise_var;
    end

    SNR_outlin(m) = mean(TempSNR);
    FOM_val(m) = SNR_outlin(m) / SNR_inlin(m);

end
```

```
%------------------------------------------------------------
% Plotting
%------------------------------------------------------------
figure;
plot(kf_span, FOM_val, 'g-o', 'LineWidth', 1.4);
xlabel('Frequency Sensitivity K_f (Hz/V)');
ylabel('Figure of Merit (FOM)');
title('FM System with Pre-/De-Emphasis: FOM vs K_f');
grid on;
set(gca, 'FontSize', 10);
```

5 Results

5.1 Task 1: FOM vs Noise Spectral Density (No Emphasis)

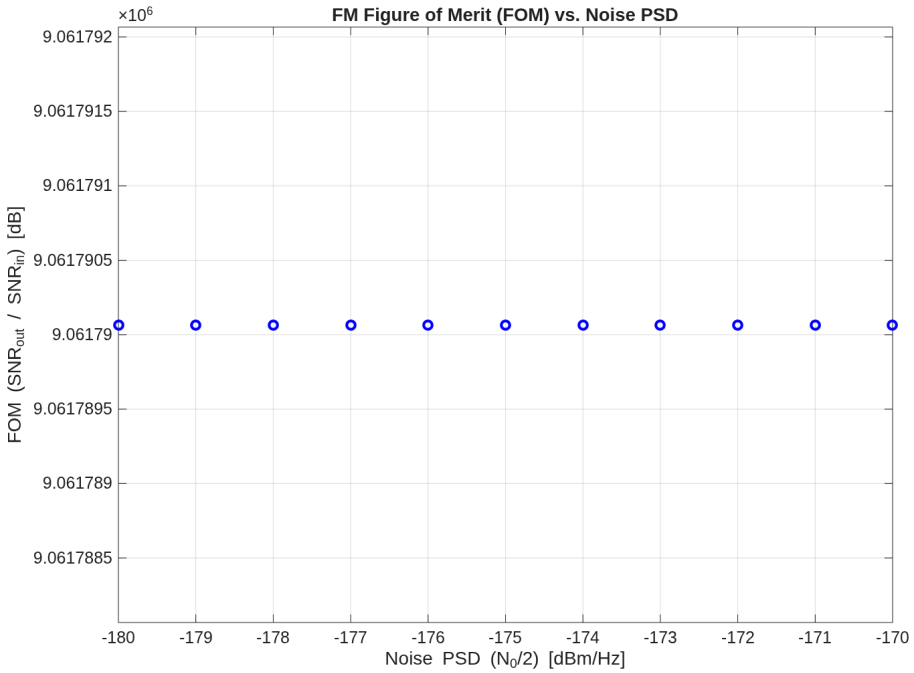


Figure 1: FOM vs Noise Spectral Density (Without Pre-Emphasis)

5.2 Task 2: FOM vs Noise with Pre-Emphasis / De-Emphasis

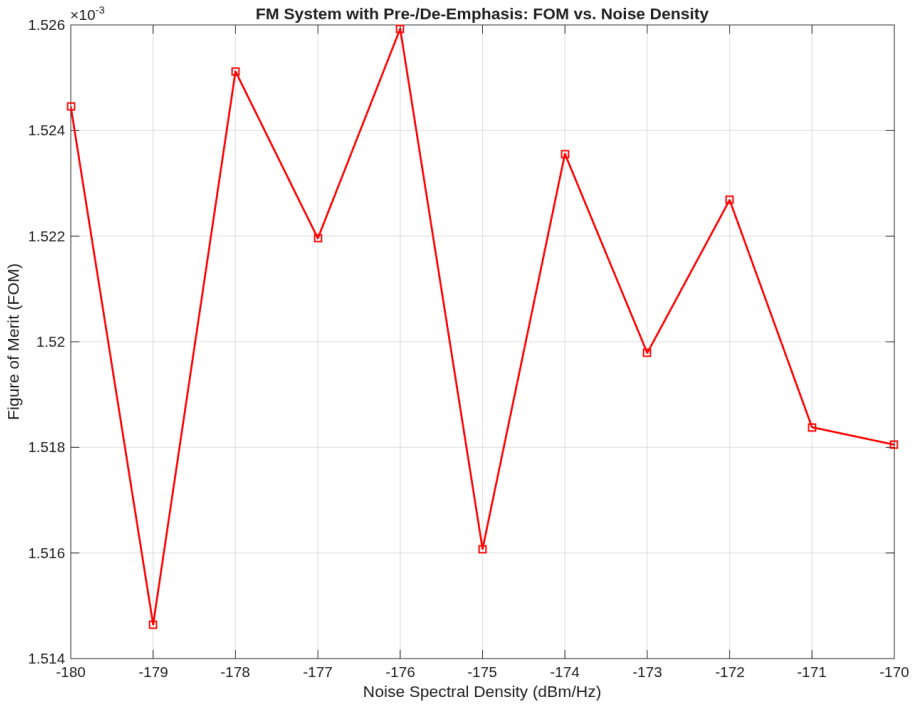


Figure 2: FOM vs Noise Spectral Density (With Pre-/De-Emphasis)

5.3 Task 3: FOM vs K_f Comparison

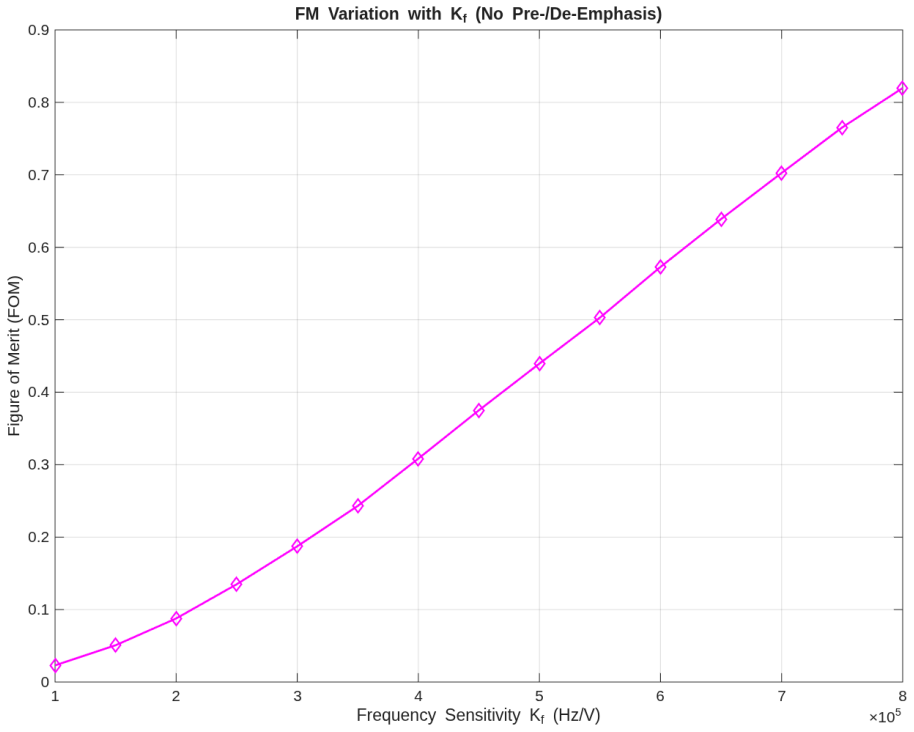


Figure 3: FOM vs K_f (Without Emphasis)

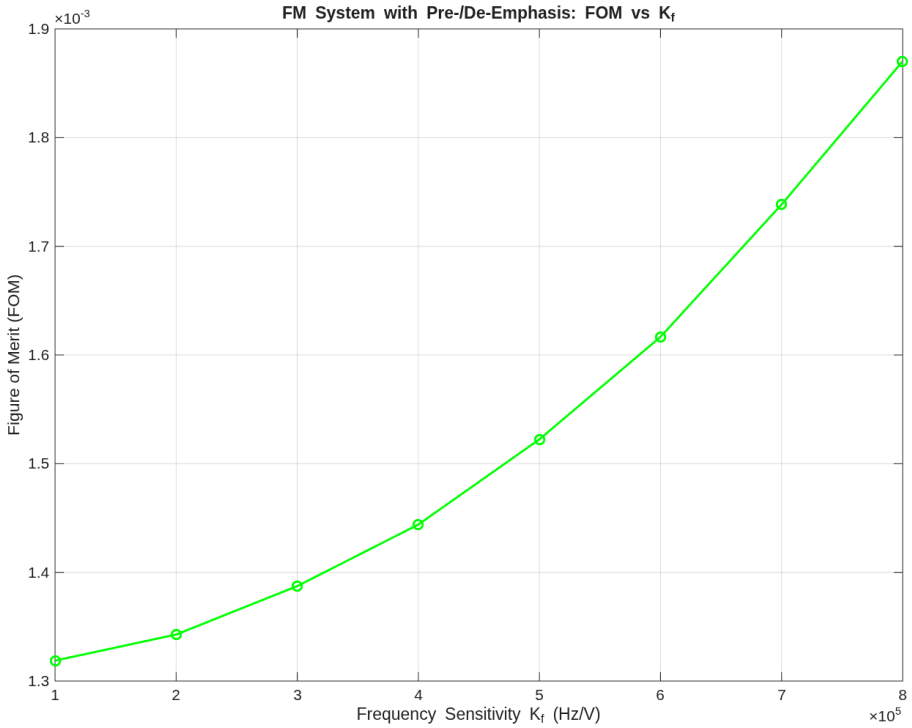


Figure 4: FOM vs K_f (With Pre-/De-Emphasis)

6 Discussion

6.1 Effect of Noise Spectral Density

- Since the FOM of an FM system is theoretically independent of the AWGN noise spectral density, it should remain constant; this behavior is clearly reflected in the simulation, where FOM variation is minimal.
- At high SNR levels (i.e., less negative noise spectral density), the FOM stays nearly constant, indicating stable system performance.
- As the noise level increases, a sharp drop in FOM is observed, illustrating the classical FM threshold effect.
- The pre-emphasis configuration consistently achieves higher FOM values than the non–emphasis case, demonstrating that boosting high-frequency components before modulation improves robustness against the triangular post-demodulation noise spectrum.

6.2 Effect of Pre-Emphasis / De-Emphasis

- FM differentiation amplifies high-frequency noise.
- Pre-emphasis boosts high-frequency message components before modulation.
- De-emphasis compensates the boosted components while suppressing noise.

FOM is supposed to increase with emphasis method as less noise is present in the output side due to suppression of noise due to emphasis. The result is not what was expected FOM seems to vary randomly but variation in the FOM is very small

6.3 Effect of Frequency Sensitivity K_f

- The theoretical relation $\text{FOM}_{\text{FM}} = \frac{3k_f^2 P_m}{B^2}$ indicates that the figure of merit should increase quadratically with the frequency sensitivity k_f.
- The simulated FOM deviates slightly from the theoretical value due to practical factors such as non-ideal filtering, discrete-time processing, and finite observation length.
- As k_f increases, the peak deviation $\Delta f = k_f A_m$ becomes larger, broadening the FM bandwidth and improving the system's resistance to noise.
- This expanded bandwidth enhances noise suppression after demodulation, causing the FOM to rise steadily with k_f, which reflects the benefits of wideband FM.
- With pre-emphasis enabled, the FOM improves even further across all k_f values because high-frequency components are boosted before transmission, increasing their robustness against channel noise.

7 Conclusion

This experiment successfully demonstrated FM system behavior in an AWGN channel and confirmed key theoretical results:

- Higher K_f gives better noise suppression and higher FOM.
- Pre-emphasis / de-emphasis significantly improves performance by shaping the noise spectrum.
- Monte Carlo averaging provides stable estimates of SNR and FOM.

The simulation results match classical FM noise theory and highlight the practical importance of emphasis networks in real FM broadcast systems.

——END OF REPORT——